



Universidad Católica Boliviana "San Pablo"
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

La Paz - Bolivia



Desarrollo de un Sistema de
Información Organizacional Web
basado en el enfoque de
Procesamiento de Transacciones
(TPS) para la gestión de procesos,
usuarios, transacciones y reportes
para una cafetería

Integrantes:

- Claros Suntura Juan Jose (*Desarrollador FullStack*)
- Lecoña Condori Elias Milan (*Desarrollador FullStack*)
- Maldonado Carvajal Alan Ariel (*Scrum Master*)
- Torrez Calle Alvaro Ariel (*Product Owner*)

Materia: Ingeniería de Software (SIS-213)

Docente: Miguel Angel Pacheco Arteaga

Fecha: 10/04/2026

Universidad: Universidad Católica Boliviana

Índice General

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS	1
1.1 Introducción	1
1.2 Antecedentes	2
1.2.1 Antecedentes del objeto de estudio	2
1.2.2 Referencias técnicas de otros sistemas TPS	3
1.3 Descripción del objeto de estudio	5
1.4 Descripción de Procesos del Sistema	7
1.4.1 Proceso 1: Autenticación y Control de Acceso	7
1.4.2 Proceso 2: Gestión del Catálogo (Administrador)	7
1.4.3 Proceso 3: Registro de Orden en el POS (Cajero)	8
1.4.4 Proceso 4: Procesamiento Transaccional y Cierre (Backend)	8
1.4.5 Proceso 5: Generación de Comprobante (Factura/Ticket)	8
1.4.6 Proceso 6: Reportes y Arqueo de Caja	8
1.5 Formulación del Problema	9
1.6 Objetivos	9
1.6.1 Objetivo General	9
1.6.2 Objetivos Específicos	9
1.7 Justificación	10
1.7.1 Justificación Técnica	10
1.7.2 Justificación Organizacional	11
1.7.3 Justificación Económica	11
1.8 Límites y Alcances	11
1.8.1 Límites	11
1.8.2 Alcances	12
1.9 Metodología del Proyecto	12
1.9.1 Tipo de estudio	12
1.9.2 Metodología de desarrollo	13

1.9.3 Técnicas	13
1.10 Análisis preliminar del sistema TPS	14
1.11 Propuesta de solución	15
1.12 Cronograma	15
Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS	18
2.1 Sistemas de Información Organizacional	18
2.1.1 Definición	18
2.1.2 Componentes	18
2.1.3 Tipos de sistemas	19
2.2 Sistema de Procesamiento de Transacciones (TPS)	19
2.2.1 Definición	19
2.2.2 Características	20
2.2.3 Funciones	21
2.2.4 Evolución hacia sistemas web	21
2.3 Arquitectura de sistemas web	22
2.3.1 Cliente	22
2.3.2 Servidor	23
2.3.3 API	23
2.3.4 Base de datos	23
2.3.5 Flujo del Sistema	23
2.4 Seguridad en sistemas de información	24
2.4.1 Autenticación	24
2.4.2 Autorización	24
2.4.3 Roles	24
2.4.4 Control de acceso	25
2.5 Base de datos	25
2.5.1 Modelo relacional	25
2.5.2 Integridad	25
2.5.3 Normalización	26

2.5.4 Transacciones	26
2.6 Metodología de desarrollo	26
2.6.1 Scrum	26
Referencias Bibliográficas	28

Índice de Figuras, Tablas y Diagramas

Figuras

1	Ejemplo de problemas en las cafeterias	2
---	--	---

Tablas

1	Cronograma de Sprints	17
---	---------------------------------	----

Diagramas

1	Diagrama de Arquitectura MERN	22
---	---	----

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS

1.1 Introducción

En el contexto actual de la transformación digital, las organizaciones de todo tamaño enfrentan la necesidad imperiosa de modernizar sus procesos operativos. Los Sistemas de Información Organizacional (SIO) han emergido como la respuesta tecnológica a esta necesidad, permitiendo a las empresas capturar, procesar y distribuir información de manera eficiente, precisa y centralizada. Dentro de esta categoría, los Sistemas de Procesamiento de Transacciones (TPS, por sus siglas en inglés — *Transaction Processing Systems*) representan el eslabón más fundamental: son la capa operativa que registra y procesa cada evento del negocio en tiempo real, constituyendo la base sobre la que se construyen todos los demás niveles de información organizacional.

La evolución histórica de los TPS es un reflejo directo del avance tecnológico. En sus orígenes, durante las décadas de 1960 y 1970, estos sistemas operaban en mainframes centralizados con procesamiento por lotes (*batch processing*), donde las transacciones se acumulaban y procesaban en diferido. Con la llegada de las redes de computadoras y, posteriormente, de Internet, los TPS evolucionaron hacia arquitecturas distribuidas de procesamiento en línea (*OLTP* — *Online Transaction Processing*), capaces de manejar miles de transacciones concurrentes con tiempos de respuesta de milisegundos. Hoy en día, la adopción de arquitecturas web modernas basadas en *stacks* como MERN (MongoDB, Express.js, React.js, Node.js) ha democratizado la implementación de TPS robustos, haciéndolos accesibles incluso para pequeñas y medianas empresas que anteriormente no podían costear soluciones de este tipo.

En este marco, el presente proyecto propone el desarrollo de un Sistema de Información Organizacional Web basado en el enfoque TPS, orientado específicamente a la gestión integral de una cafetería en la ciudad de La Paz, Bolivia. El sistema, concebido como un Punto de Venta (*Point of Sale* — POS) web, tiene por objetivo digitalizar y centralizar los procesos críticos del negocio: la toma y seguimiento de órdenes por mesa,

la administración del catálogo de productos, el control de acceso diferenciado por roles de usuario, el procesamiento de cobros y la generación automatizada de reportes financieros. La solución busca reemplazar los procesos manuales actualmente vigentes — basados en libretas de papel, cálculos mentales y ausencia total de trazabilidad — por una plataforma tecnológica accesible, segura y escalable que eleve la eficiencia operativa del establecimiento.

1.2 Antecedentes

1.2.1 Antecedentes del objeto de estudio

El objeto de estudio del presente proyecto es una cafetería de tamaño mediano en la ciudad de La Paz, Bolivia, que actualmente opera sus procesos de atención y venta de manera completamente manual. Este escenario, lejos de ser un caso aislado, refleja la realidad predominante en el sector gastronómico local, donde la adopción de tecnologías de gestión sigue siendo incipiente.



Figura 1: Ejemplo de problemas en las cafeterías

En su estado actual, el flujo operativo de la cafetería depende íntegramente de la intervención humana no sistematizada: los pedidos de los clientes son anotados a mano por el personal en libretas o talonarios de papel, la comunicación entre el área de atención y la barra de preparación se realiza de forma verbal o mediante la entrega física de las comandas, y el proceso de cobro se ejecuta mediante cálculos mentales o con

el apoyo de calculadoras de escritorio. Al cierre de cada jornada, el arqueo de caja se realiza de forma manual, contrastando el efectivo físico con las anotaciones del día, un proceso propenso a errores y discrepancias.

Esta situación genera un conjunto de problemas operativos concretos y recurrentes que afectan tanto la eficiencia del servicio como la salud financiera del negocio:

- **Pérdida e ilegibilidad de comandas:** El registro manual en papel está expuesto a extravíos, manchas o escritura ilegible, lo que deriva en errores en la preparación de pedidos y conflictos con los clientes.
- **Ausencia de control y auditoría de usuarios:** Al no existir un sistema de registro, resulta imposible determinar qué miembro del personal procesó, modificó o anuló una orden, eliminando cualquier posibilidad de rendición de cuentas.
- **Descuadres de caja recurrentes:** El cobro basado en cálculo mental o en calculadoras simples, sin un sistema que valide automáticamente los totales, genera discrepancias diarias entre los ingresos registrados y el efectivo real.
- **Nula trazabilidad histórica:** La ausencia de una base de datos implica que no existe ningún registro histórico de ventas. La gerencia no puede conocer cuáles son los productos más vendidos, los picos de demanda por hora o los ingresos acumulados por período, privándose de información crítica para la toma de decisiones estratégicas.

1.2.2 Referencias técnicas de otros sistemas TPS

Como parte del análisis de referencia, se relevaron tres sistemas de gestión para cafeterías y restaurantes disponibles públicamente en GitHub. El estudio de estos proyectos permite identificar patrones de diseño comunes, tecnologías adoptadas por la comunidad y brechas funcionales que el presente sistema busca superar.

1. proyecto-cafeteria — ValentinHer (GitHub)

- **Repositorio:** <https://github.com/ValentinHer/proyecto-cafeteria.git>
- **Descripción general:** Sistema de gestión para cafetería desarrollado como

proyecto académico. Implementa las operaciones básicas de un punto de venta: registro de productos, toma de pedidos y generación de órdenes.

- **Stack tecnológico:** Aplicación web con arquitectura cliente-servidor, base de datos relacional para la persistencia de productos y transacciones, e interfaz de usuario orientada a la administración del negocio.
- **Funcionalidades identificadas:** Gestión del catálogo de productos (CRUD), registro de pedidos por mesa, y consulta de historial de ventas a nivel básico.
- **Diferencia con el sistema propuesto:** Este proyecto carece de un sistema de autenticación basado en roles diferenciados (Administrador vs. Cajero), no implementa procesamiento transaccional ACID para garantizar la integridad de los cobros concurrentes, y no genera comprobantes de pago en formato PDF. El sistema propuesto aborda estas limitaciones mediante JWT, sesiones de transacción en MongoDB y el módulo de facturación con PDFKit/jsPDF.

2. Sistema-POS-Restaurantes — ValentinPacheco (GitHub)

- **Repositorio:** <https://github.com/ValentinPacheco/Sistema-POS-Restaurantes.git>
- **Descripción general:** Sistema de Punto de Venta orientado a restaurantes, con enfoque en la gestión de órdenes en sala y el flujo de cobro al cliente. Representa una solución más cercana al dominio del presente proyecto por su naturaleza POS.
- **Stack tecnológico:** Implementación web con separación de capas frontend y backend, manejo de estado de mesas y control de órdenes activas.
- **Funcionalidades identificadas:** Panel de estado de mesas, creación y modificación de órdenes en curso, cálculo de totales y cierre de cuenta por mesa.
- **Diferencia con el sistema propuesto:** Si bien aborda la gestión de mesas y órdenes, el sistema no contempla una arquitectura de microservicios contenerizada con Docker, ni un despliegue en infraestructura

cloud (AWS/DigitalOcean). Asimismo, el control de acceso por roles y la generación automatizada de reportes financieros con cortes de caja son funcionalidades ausentes que el presente proyecto incorpora de forma nativa.

3. **cafeteria-app** — FFW4 (GitHub)

- **Repositorio:** <https://github.com/FFW4/cafeteria-app.git>
- **Descripción general:** Aplicación web para la gestión de una cafetería, con foco en la experiencia del operador en el punto de atención. Desarrollada con un enfoque pragmático orientado a la agilidad en la toma de pedidos.
- **Stack tecnológico:** Aplicación de página única (*SPA*) con componentes reactivos para la interfaz del POS y conexión a un servicio de backend para la persistencia de datos.
- **Funcionalidades identificadas:** Selección de productos por categoría, armado del carrito de compras, asignación de pedidos y registro de ventas.
- **Diferencia con el sistema propuesto:** Esta aplicación no implementa autenticación segura mediante *tokens* JWT ni encriptación de contraseñas con *bcrypt*, exponiendo el sistema a vulnerabilidades de acceso no autorizado. Tampoco cuenta con un módulo de reportes analíticos ni con transacciones ACID multi-documento que garanticen la inmutabilidad de los registros históricos de venta, aspectos críticos en un TPS de producción.

1.3 Descripción del objeto de estudio

La unidad de análisis del presente proyecto es una cafetería de servicio rápido ubicada en la ciudad de La Paz, Bolivia, con capacidad para atender simultáneamente a múltiples mesas. El establecimiento ofrece un menú compuesto principalmente por bebidas calientes y frías (café, infusiones, batidos) y una selección de alimentos de preparación rápida (postres, sándwiches, snacks), atendiendo a una clientela diversa en horario continuo.

Estructura organizacional y actores del sistema:

El personal operativo del establecimiento se organiza en dos roles funcionales claramente diferenciados, que se corresponden directamente con los actores del sistema a desarrollar:

- **Administrador:** Responsable de la gobernanza general del negocio. Gestiona el catálogo de productos (altas, bajas y modificaciones de precios), administra las cuentas del personal operativo y tiene acceso a los reportes de ventas e indicadores financieros.
- **Cajero / Operador POS:** Personal de atención directa al cliente. Su función central es construir y procesar órdenes en la interfaz del punto de venta, asignarlas a la mesa correspondiente y ejecutar el cobro al momento de cerrar la cuenta.

Flujo actual del negocio (situación sin sistema):

El ciclo de servicio actual sigue el siguiente flujo manual: el cliente se ubica en una mesa disponible → el cajero toma el pedido verbalmente y lo anota en la comanda de papel → la comanda se entrega en barra para preparación → los productos son entregados al cliente → al solicitar la cuenta, el cajero suma manualmente los ítems, comunica el total y recibe el pago en efectivo → el dinero se deposita en la caja registradora mecánica.

Flujo propuesto del negocio (con el sistema TPS):

Con la implementación del sistema, el flujo se transforma radicalmente: el cajero selecciona los productos del menú digital interactivo y los asigna a la mesa del cliente en la interfaz POS → el sistema calcula automáticamente el subtotal en tiempo real → al confirmar el cobro, el *backend* procesa matemáticamente la transacción, aplica impuestos y sella el registro de forma inmutable en la base de datos → el sistema genera automáticamente el comprobante de pago (ticket/factura) → la mesa queda marcada como disponible para el próximo cliente. Cada uno de estos eventos queda registrado con fecha, hora, cajero responsable y detalle de productos, garantizando trazabilidad completa y eliminando cualquier posibilidad de descuadre.

1.4 Descripción de Procesos del Sistema

Esta sección describe la forma en que el Sistema TPS-POS será implementado operativamente, detallando los procesos clave que lo componen y cómo cada uno se ejecuta dentro de la arquitectura MERN propuesta.

1.4.1 Proceso 1: Autenticación y Control de Acceso

Al ingresar al sistema, el operador introduce sus credenciales (usuario y contraseña) en la pantalla de *login* construida con React.js. El *frontend* envía una solicitud HTTP POST al *endpoint* `/api/auth/login` del servidor Node.js/Express.js. El *backend* recupera el registro del usuario desde MongoDB, verifica la contraseña mediante la función de comparación de *bcrypt* y, si es válida, genera un JSON Web Token (JWT) firmado que incluye en su *payload* el identificador del usuario y su rol (Administrador o Cajero). Este *token* es devuelto al cliente y almacenado en el estado global de Redux, siendo adjuntado automáticamente en la cabecera *Authorization* de todas las peticiones subsiguientes. Los *middlewares* de Express validan el *token* en cada ruta protegida antes de permitir el acceso a los recursos.

1.4.2 Proceso 2: Gestión del Catálogo (Administrador)

El Administrador accede al módulo de gestión de productos desde su panel exclusivo. A través de formularios React, puede crear, editar, activar o desactivar ítems del menú (nombre, precio, categoría, imagen). Cada acción dispara una petición REST al *backend* (POST, PUT o PATCH según corresponda), que valida los datos contra el esquema Mongoose de la colección *Productos* en MongoDB antes de persistir el cambio. Las modificaciones de precio no alteran registros históricos: las órdenes ya cerradas conservan el precio exacto del momento de la venta gracias a la desnormalización del carrito (*cartItems*).

1.4.3 Proceso 3: Registro de Orden en el POS (Cajero)

El Cajero opera la interfaz POS táctil. Selecciona una mesa disponible del panel de estados, luego elige productos del menú interactivo organizado por categorías. Cada selección actualiza el estado del carrito en Redux, recalculando subtotales y totales en tiempo real sin consultar el servidor. Una vez completada la orden, el Cajero confirma el método de pago. El *frontend* envía la orden completa (mesa, cajero, *cartItems* con precios actuales, total calculado) al *endpoint* `/api/orders` del *backend*.

1.4.4 Proceso 4: Procesamiento Transaccional y Cierre (Backend)

Al recibir la orden, el *backend* inicia una Sesión de Transacción de MongoDB para garantizar las propiedades ACID. Dentro de la transacción atómica se ejecutan en secuencia: (1) validación matemática del total enviado por el cliente, (2) inserción del documento de la orden en la colección Facturas/Órdenes con todos los datos inmutables, (3) actualización del estado de la Mesa asignada de “ocupada” a “disponible”. Si cualquiera de estos pasos falla (por ejemplo, por un corte de red), MongoDB ejecuta un *Rollback* completo, garantizando que no queden “ventas a medias” en la base de datos.

1.4.5 Proceso 5: Generación de Comprobante (Factura/Ticket)

Una vez confirmada la transacción, el *backend* devuelve los datos de la orden sellada al *frontend*. React activa automáticamente la opción de generar el comprobante, invocando el módulo de facturación (PDFKit o jsPDF) que construye el ticket en formato PDF con el detalle de los ítems, el total cobrado, la mesa, el cajero y la fecha exacta. El comprobante queda disponible para impresión inmediata desde el navegador.

1.4.6 Proceso 6: Reportes y Arqueo de Caja

El Administrador accede al módulo de reportes para consultar el resumen financiero del turno o del período seleccionado. El *backend* ejecuta consultas de agregación (*Ag-*

gregation Pipeline) sobre la colección de órdenes en MongoDB, consolidando ingresos totales, desglose por método de pago, productos más vendidos y ventas por cajero. Los resultados son renderizados en tablas y gráficos en el *frontend* de React, permitiendo al Administrador realizar el arqueo de caja con datos exactos y auditables.

1.5 Formulación del Problema

¿Cómo el desarrollo de un prototipo funcional basado en el procesamiento de transacciones permite proyectar una reducción en los errores operativos y una mejora en el control de usuarios por roles, asegurando la consistencia de los datos financieros desde una arquitectura web?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un prototipo funcional de un Sistema de Información Organizacional Web basado en el enfoque de Procesamiento de Transacciones (TPS), que permita gestionar procesos, usuarios, transacciones y reportes mediante una arquitectura *backend* funcional, con el propósito de proyectar una reducción en los errores operativos y una mejora en el control de usuarios por roles, asegurando la consistencia de los datos financieros desde una arquitectura web.

1.6.2 Objetivos Específicos

- Implementar un módulo de gestión de pedidos en tiempo real que permita a los meseros registrar, modificar y hacer seguimiento del estado de las órdenes (en preparación, servido, pagado) mediante una interfaz táctil dinámica construida con React.js.
- Diseñar e integrar un sistema de control de acceso basado en roles (RBAC) con autenticación segura mediante JSON Web Tokens (JWT), diferenciando los permisos y vistas del Administrador, el Cajero y el Mesero.

- Desarrollar un módulo de administración visual de mesas y reservas que presente un panel de estados en tiempo real, permitiendo a los operadores conocer la disponibilidad y el estado de cada mesa del establecimiento.
- Construir las APIs RESTful del *backend* con Node.js y Express.js, conectadas a una base de datos MongoDB, para soportar todas las operaciones CRUD de los módulos de productos, órdenes, mesas y usuarios.
- Implementar un módulo de facturación y generación de recibos que automatice el cálculo del cobro total y produzca comprobantes detallados en formato PDF, integrando métodos de pago simulados mediante una pasarela estándar.
- Desplegar el sistema en infraestructura *cloud* (AWS o DigitalOcean) utilizando contenedores Docker para garantizar la portabilidad, disponibilidad y escalabilidad del entorno productivo.

1.7 Justificación

1.7.1 Justificación Técnica

El desarrollo de este prototipo se fundamenta en la implementación de una arquitectura web moderna y de alta disponibilidad, estructurada bajo el entorno Cliente-Servidor empleando el stack MERN. Esta elección garantiza una separación absoluta entre la interfaz de usuario y la lógica de negocio. Se construirá un backend funcional y asíncrono utilizando Node.js y Express.js, optimizado para el procesamiento rápido de múltiples transacciones concurrentes (TPS). Para la persistencia de datos, se utilizará MongoDB, una base de datos orientada a documentos que otorga la flexibilidad necesaria para manejar carritos de compra dinámicos y catálogos escalables. En términos de seguridad, el sistema prescindirá del manejo tradicional de sesiones, implementando en su lugar autenticación sin estado mediante JSON Web Tokens (JWT) y el cifrado irreversible de contraseñas con el algoritmo Bcrypt. Esta infraestructura dota al sistema de una alta escalabilidad técnica, permitiendo futuras integraciones o el despliegue en infraestructuras en la nube (Cloud Hosting) sin necesidad de refactorizar el código base.

1.7.2 Justificación Organizacional

A nivel organizacional, el prototipo representa un salto cualitativo en la gestión de procesos internos de la cafetería. La automatización de transacciones permite estandarizar el ciclo de servicio, eliminando las ambigüedades y retrasos propios de los procesos manuales y la comunicación verbal. Se establecerá un estricto control de usuarios mediante la implementación de un modelo de acceso basado en roles (RBAC), asegurando que los operadores (cajeros) y los tomadores de decisiones (administradores) interactúen únicamente con las interfaces y datos correspondientes a sus responsabilidades. Además, la capacidad del sistema para la generación de reportes automáticos dotará a la gerencia de información estructurada y en tiempo real sobre el rendimiento del negocio, transformando datos aislados en conocimiento estratégico.

1.7.3 Justificación Económica

La viabilidad y necesidad del proyecto se sustentan en el impacto financiero positivo que genera la digitalización. La implementación del TPS asegura una drástica reducción de errores operativos; al automatizar el cálculo de subtotales, impuestos y cambio, se eliminan los recurrentes descuadres de caja diarios. Esto se traduce inmediatamente en la prevención de fugas de capital. Asimismo, la optimización del tiempo por cada transacción permite aumentar la capacidad de atención en horas pico (mayor rotación de mesas), elevando los ingresos. A mediano plazo, estas mejoras operativas confluyen en una notable reducción de costos operativos y administrativos.

1.8 Límites y Alcances

1.8.1 Límites

El alcance del presente proyecto se enmarca dentro de las siguientes restricciones técnicas y operativas: * El sistema será exclusivamente web, requiriendo un navegador moderno y conexión a la red local o internet para su funcionamiento. * Se utilizará una base de datos NoSQL (MongoDB) en lugar de una base de datos relacional tradicional,

priorizando la agilidad en la estructura de los pedidos (documentos). * Tendrá un sistema de autenticación de usuarios interno y cerrado; no se permitirá el registro público ni el inicio de sesión con redes sociales. * No incluirá integración con sistemas externos en esta fase (como plataformas de delivery de terceros, o pasarelas de impuestos gubernamentales directas).

1.8.2 Alcances

El sistema funcional entregado al finalizar el proyecto permitirá realizar las siguientes operaciones: * Gestionar procesos operativos centrales, incluyendo la asignación y liberación de mesas de la cafetería. * Gestionar usuarios y roles, otorgando permisos específicos para la administración del menú o la operación de la caja. * Registrar transacciones comerciales en tiempo real, almacenando el detalle exacto de cada venta de manera inmutable. * Generar reportes financieros e históricos de ventas filtrados por fechas o turnos. * Administrar información maestra del negocio, como el alta, baja y modificación de productos, precios y categorías del catálogo.

1.9 Metodología del Proyecto

1.9.1 Tipo de estudio

La presente investigación se define bajo tres enfoques metodológicos: * **Aplicado:** Ya que no se limita a la investigación teórica, sino que busca resolver un problema operativo concreto de la cafetería mediante la construcción de una herramienta útil. * **Tecnológico:** Debido a que el núcleo de la solución requiere la aplicación de ingeniería de software, lenguaje de programación y arquitecturas de sistemas modernos. * **Descriptivo:** Puesto que requiere analizar, detallar y comprender la naturaleza del flujo de trabajo actual de la organización para poder traducirlo a requerimientos de software.

1.9.2 Metodología de desarrollo

Para la construcción del software se adoptará **Scrum**, un marco de trabajo ágil iterativo e incremental, ideal para proyectos tecnológicos donde los requerimientos pueden evolucionar. * **Sprint**: El desarrollo se dividirá en ciclos de trabajo cortos y de duración fija (ej. dos semanas), garantizando revisiones periódicas. * **Product Backlog**: Se manejará una lista priorizada con todos los requerimientos funcionales, técnicos y de interfaz que el sistema POS necesita para operar. * **Sprint Backlog**: En cada planificación, el equipo seleccionará un subconjunto de tareas del Product Backlog para diseñarlas, codificarlas y probarlas durante ese ciclo. * **Entregables**: Al finalizar cada Sprint, se presentará un incremento de software funcional (un módulo operativo, como el panel de login o la interfaz de toma de pedidos) listo para ser validado por los interesados.

1.9.3 Técnicas

Para garantizar que el sistema capture fielmente la realidad del negocio y se diseñe con robustez técnica, se aplicarán las siguientes técnicas:

- **Entrevistas**: Se realizarán entrevistas estructuradas a los actores clave (el administrador y el personal de caja). El objetivo es extraer requerimientos funcionales precisos, comprender los cuellos de botella actuales en la toma de pedidos y definir las métricas que la gerencia necesita visualizar en los reportes diarios.
- **Observación**: Se aplicará la observación directa no participante durante las horas pico de la cafetería. Esta técnica es fundamental para mapear el flujo de trabajo real (tiempo de atención por cliente, comunicación entre caja y preparación, y manejo físico del efectivo), identificando fricciones operativas que las entrevistas podrían omitir.
- **Análisis documental**: Se examinarán los registros físicos actuales de la organización (comandas de papel, libretas de contabilidad, inventarios manuales y facturas de proveedores). El análisis de estos documentos es el paso previo a la

normalización de datos, permitiendo diseñar los esquemas exactos que conformarán la base de datos (MongoDB).

- **Modelado UML:** Se utilizará el Lenguaje Unificado de Modelado como puente entre la recolección de datos y la codificación. Se elaborarán Diagramas de Casos de Uso para definir la interacción por roles; Diagramas de Actividades para trazar el flujo lógico del Procesamiento de Transacciones (desde el pedido hasta la factura); y Diagramas de Clases para estructurar las entidades del sistema.
- **Modelo C4 (Técnica de Arquitectura):** Complementando a UML, se utilizará el modelo C4 para documentar la arquitectura web. Se elaborarán diagramas de Contexto (el sistema en su entorno), Contenedores (interacción entre React, Node.js y MongoDB), Componentes (controladores y servicios) y Código, facilitando la comprensión técnica del prototipo.

1.10 Análisis preliminar del sistema TPS

El relevamiento inicial demuestra que los **procesos actuales** de la cafetería son enteramente manuales. El ciclo de servicio se basa en la transcripción de pedidos en libretas, cálculos matemáticos mentales y arqueos de caja basados en anotaciones físicas. Los **problemas** derivados de esta operatividad son severos: pérdida constante de comandos, errores humanos en el cobro, lentitud en el servicio y una total falta de auditoría que resulta en discrepancias financieras.

En cuanto a los **usuarios**, el personal opera sin una jerarquía digital; cualquiera puede modificar o anular un registro físico sin dejar rastro. Las **transacciones**, que deberían ser tratadas como eventos inmutables de entrada de capital, carecen de respaldo. Finalmente, la **información** generada por el negocio se pierde al finalizar el día, privando a la gerencia de datos históricos críticos para analizar qué productos se venden más o en qué horarios se requiere más personal.

1.11 Propuesta de solución

Para resolver la problemática descrita, se propone el desarrollo y despliegue de un **Sistema de Información Organizacional Web**, específicamente orientado como un Punto de Venta (POS) transaccional.

- **Arquitectura:** El sistema se construirá bajo el patrón Cliente — Servidor, separando la capa de presentación visual de la capa de procesamiento lógico y acceso a datos, garantizando rendimiento y seguridad.
- **Tecnologías seleccionadas:**
 - **Backend (Servidor y Lógica):** Se desarrollará en **Node.js** utilizando el framework Express.js, creando una API RESTful encargada de la validación matemática, autenticación de usuarios y reglas de negocio.
 - **Base de datos (Persistencia):** Se utilizará **MongoDB** (NoSQL), modelada mediante Mongoose, para almacenar de forma ágil y estructurada los usuarios, catálogos y el historial inmutable de órdenes de venta.
 - **Frontend (Cliente y UI):** Se construirá con la biblioteca **React.js** (junto con HTML, CSS y JavaScript), desarrollando una interfaz de usuario interactiva, dinámica y adaptada a pantallas táctiles, que permita a los cajeros operar con máxima velocidad y mínima fricción.

1.12 Cronograma

El proyecto tiene una duración total de **4 meses (16 semanas)**, organizado en 8 *Sprints* de 2 semanas cada uno bajo el marco Scrum.

Sprint	Semanas	Fase	Actividades principales	Entregable
0	1–2	Inicio y Diseño	Levantamiento de requerimientos, diseño de <i>wireframes</i> UI/UX, modelado de base de datos, configuración del repositorio GitHub y entorno Docker.	<i>Product Backlog</i> , diseño de BD, <i>wireframes</i> aprobados.
1	3–4	<i>Backend</i> – Fundamentos	Configuración de Express.js, conexión MongoDB con Mongoose, modelos de datos (User, Product, Table, Order), sistema de autenticación JWT y <i>middleware</i> de roles.	API de autenticación funcional (registro, <i>login</i> , roles).
2	5–6	<i>Backend</i> – Módulos Core	APIs RESTful para gestión de productos del menú (CRUD), gestión de mesas (CRUD + estados) y gestión de órdenes (crear, actualizar estado).	<i>Endpoints</i> de Products, Tables y Orders documentados.
3	7–8	<i>Frontend</i> – Base y Auth	Configuración de React.js + Redux Toolkit + React Router, pantallas de <i>Login</i> , <i>layout</i> principal y conexión con el API de autenticación.	<i>Frontend</i> base funcional con <i>login</i> y roles.
4	9–10	<i>Frontend</i> – POS	Módulo POS táctil: selección de categorías, productos, cantidad y adición al carrito de órdenes; envío de pedidos al <i>backend</i> .	Interfaz POS funcional conectada al <i>backend</i> .

Sprint	Semanas	Fase	Actividades principales	Entregable
5	11–12	Mesas y Dash-board	Panel visual de estados de mesas, módulo de administración de menú (alta/baja de productos), vista de órdenes activas para cocina.	Gestión de mesas e interfaz de cocina operativa.
6	13–14	Facturación y Pagos	Módulo de generación de facturas en PDF, cálculo automático de totales, integración de métodos de pago simulados, historial de ventas.	Facturación y cierre de órdenes completo.
7	15–16	Cierre: QA y Despliegue	Pruebas funcionales e integración (QA), corrección de <i>bugs</i> , despliegue en AWS/DigitalOcean con Docker, documentación técnica final y capacitación al usuario.	Sistema desplegado en producción y manual de usuario.

Tabla 1: Cronograma de Sprints

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS

2.1 Sistemas de Información Organizacional

2.1.1 Definición

Un Sistema de Información Organizacional (SIO) es un conjunto integrado de componentes — personas, procesos, datos, *hardware* y *software* — diseñado para recolectar, almacenar, procesar y distribuir información que apoye la coordinación, el control, el análisis y la toma de decisiones dentro de una organización (Laudon & Laudon, 2020). A diferencia de un simple programa informático, un SIO está profundamente imbricado con los procesos de negocio de la organización: define cómo fluye la información entre los actores, cuándo se captura, cómo se transforma y quién tiene acceso a ella.

En términos más precisos, un SIO transforma datos crudos (ej. el registro de una venta) en información significativa y estructurada (ej. un reporte de ingresos diarios), que a su vez se convierte en conocimiento útil para la gestión (ej. la identificación del turno de mayor rentabilidad). Este proceso de transformación es el núcleo del valor que aportan los SIO a las organizaciones modernas.

2.1.2 Componentes

Todo Sistema de Información Organizacional se articula en torno a seis componentes fundamentales que trabajan de forma interdependiente:

- **Hardware:** La infraestructura física que sustenta el sistema: servidores, terminales de trabajo, dispositivos de red y, en el contexto del presente proyecto, las estaciones de trabajo desde las que el personal operará la interfaz POS.
- **Software:** Los programas y aplicaciones que procesan los datos. Incluye tanto el *software* de sistema (sistema operativo, entorno de ejecución Node.js) como el *software* de aplicación desarrollado a medida (la plataforma POS web).
- **Datos:** La materia prima del sistema. En el contexto de la cafetería, los datos

son las órdenes, los productos, los usuarios, las mesas y las transacciones que el sistema captura y persiste en la base de datos MongoDB.

- **Redes y telecomunicaciones:** La infraestructura de conectividad que permite el acceso concurrente al sistema desde múltiples dispositivos, habilitado por la arquitectura cliente-servidor del proyecto.
- **Procedimientos:** Los protocolos y flujos de trabajo que definen cómo deben interactuar los usuarios con el sistema (ej. el proceso de apertura de turno, la toma de una orden, el cierre de caja).
- **Recursos humanos:** Los actores que operan el sistema. En el proyecto, esto comprende al Administrador y al Cajero, cada uno con roles y permisos claramente delimitados.

2.1.3 Tipos de sistemas

Desde una perspectiva funcional, los SIO se clasifican en distintos tipos según el nivel organizacional al que sirven. Los **Sistemas de Procesamiento de Transacciones (TPS)** operan en el nivel operativo, capturando y procesando las transacciones cotidianas del negocio. Los **Sistemas de Información Gerencial (MIS)** consolidan la información del TPS para generar reportes estructurados destinados a la gerencia media. Los **Sistemas de Soporte a Decisiones (DSS)** asisten en la toma de decisiones complejas mediante análisis de datos y modelos. Los **Sistemas de Información Ejecutiva (EIS)** proveen información estratégica de alto nivel a los directivos. El presente proyecto se enfoca en la implementación de un TPS, que actúa como la base de toda esta pirámide informacional.

2.2 Sistema de Procesamiento de Transacciones (TPS)

2.2.1 Definición

Un Sistema de Procesamiento de Transacciones es un tipo especializado de SIO diseñado para capturar, procesar, validar y almacenar las transacciones operativas de una organización de forma inmediata, confiable y a gran escala (O'Brien & Marakas, 2011).

En el contexto del negocio, una **transacción** se define como cualquier evento discreto que modifica el estado de los datos del sistema y que debe quedar registrado de forma permanente e inalterable: el registro de una venta, la creación de una orden, el cobro de una cuenta o la modificación del catálogo de productos.

2.2.2 Características

Los TPS se distinguen de otros tipos de sistemas de información por un conjunto de atributos técnicos y funcionales que los hacen aptos para el procesamiento operativo de alto volumen:

- **Procesamiento en tiempo real (OLTP):** A diferencia del procesamiento por lotes (*batch*), los TPS modernos procesan cada transacción en el instante en que se produce, actualizando la base de datos de forma inmediata y reflejando el estado actual del negocio en todo momento.
- **Alta confiabilidad y disponibilidad:** Un TPS para un punto de venta debe estar disponible durante todo el horario operativo del negocio. La indisponibilidad del sistema implica la parálisis del servicio al cliente.
- **Integridad transaccional (propiedades ACID):** Toda transacción en un TPS debe cumplir las propiedades de Atomicidad (la transacción se ejecuta completa o no se ejecuta), Consistencia (el sistema pasa de un estado válido a otro estado válido), Aislamiento (las transacciones concurrentes no interfieren entre sí) y Durabilidad (una transacción confirmada persiste incluso ante fallos del sistema) (Elmasri & Navathe, 2015).
- **Manejo de alto volumen de datos estandarizados:** Los TPS están optimizados para procesar grandes cantidades de transacciones simples y repetitivas de forma eficiente, a diferencia de los sistemas analíticos que trabajan con consultas complejas sobre datos históricos.

2.2.3 Funciones

En el contexto específico del presente proyecto, el TPS ejecuta el siguiente ciclo funcional para cada transacción de venta:

1. **Captura de datos de origen:** El cajero construye la orden seleccionando productos del catálogo digital y asignándola a una mesa, introduciendo los datos de la transacción en el sistema mediante la interfaz POS de React.js.
2. **Validación y verificación:** El *backend* (Node.js/Express.js) verifica que el usuario tenga los permisos necesarios (validación JWT), que los ítems existan en el catálogo activo y que la mesa esté disponible.
3. **Procesamiento matemático:** El motor transaccional calcula automáticamente los subtotales por ítem, aplica los impuestos correspondientes y determina el total a cobrar, eliminando el margen de error del cálculo manual.
4. **Actualización de la base de datos:** La transacción se escribe de forma atómica en MongoDB, vinculando el documento de la orden con el cajero responsable, la mesa asignada y los ítems del carrito con sus precios exactos en ese instante.
5. **Emisión del comprobante:** El sistema genera el ticket o factura en formato PDF, disponible para impresión inmediata, y actualiza el estado de la mesa a “disponible”.

2.2.4 Evolución hacia sistemas web

Los TPS han recorrido un largo camino desde las terminales monolíticas de los años setenta. La adopción de arquitecturas web modernas —como la empleada en este proyecto— representa la fase más reciente de esta evolución, caracterizada por tres ventajas fundamentales: **ubicuidad** (el sistema es accesible desde cualquier dispositivo con navegador web en la red local del negocio), **centralización** (todos los datos residen en un único repositorio en la nube, eliminando la dispersión de información), y **escalabilidad** (la arquitectura basada en microservicios y contenedores Docker permite escalar el sistema horizontalmente para absorber incrementos en la carga de trabajo sin rediseñar la arquitectura base).

2.3 Arquitectura de sistemas web

La arquitectura del sistema POS se basa en el modelo Cliente–Servidor, una de las estructuras más utilizadas en aplicaciones web modernas por su capacidad de separación de responsabilidades, escalabilidad y mantenimiento (Sommerville, 2015). Siguiendo este diagrama:

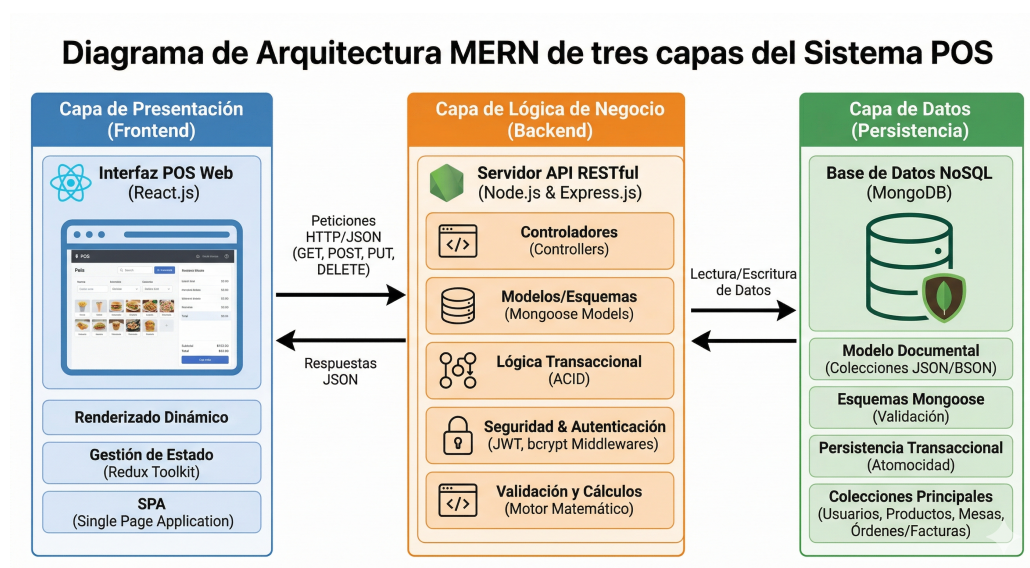


Diagrama 1: Diagrama de Arquitectura MERN

2.3.1 Cliente

El cliente representa la capa de presentación del sistema, encargada de interactuar directamente con el usuario final mediante una interfaz gráfica accesible desde el navegador. En este proyecto, el cliente será desarrollado utilizando React.js, permitiendo:

- Renderizado dinámico de componentes (Virtual DOM).
- Interacciones en tiempo real en el POS.
- Manejo del estado global mediante Redux Toolkit.
- Navegación sin recarga de página (SPA).

Funciones principales:

- Capturar datos de entrada (pedidos, login).

- Mostrar información procesada.
- Enviar solicitudes HTTP al servidor.

2.3.2 Servidor

El servidor constituye la capa de lógica de negocio. Tecnologías: Node.js; Express.js.

Funciones principales:

- Procesamiento de órdenes.
- Validaciones y cálculos.
- Ejecución de la lógica transaccional.

2.3.3 API

La API permite la comunicación entre cliente y servidor mediante HTTP y JSON. Actúa como el puente documentado que estructura y transmite la información bidireccionalmente. Características:

- Métodos estándar: GET, POST, PUT, DELETE.
- Arquitectura RESTful.

2.3.4 Base de datos

Repositorio central donde reposa la persistencia de las entidades. Es el componente responsable de almacenar los datos operacionales a largo plazo para asegurar la durabilidad y disponibilidad de la información de las ventas y el menú.

2.3.5 Flujo del Sistema

1. Usuario interactúa con frontend.
2. Cliente envía petición HTTP a la API.
3. Servidor procesa la solicitud y valida.
4. Se consulta o impacta la base de datos.
5. Servidor responde en JSON.

6. Frontend actualiza la interfaz.

2.4 Seguridad en sistemas de información

Como modelador y encargado de la seguridad arquitectónica, se establece que un sistema de ventas (POS) debe proteger de forma absoluta sus *endpoints* (rutas de API) y la persistencia de datos.

2.4.1 Autenticación

Se descartan las sesiones tradicionales. El sistema implementa JSON Web Tokens (JWT). Tras validar credenciales (contraseñas previamente procesadas con funciones criptográficas unidireccionales de *hash*, como *bcrypt*), el *backend* emite un token firmado (Stallings, 2017). Este token viaja en las cabeceras HTTP de cada petición del cliente, garantizando que el usuario es quien dice ser sin consultar la base de datos reiteradamente.

2.4.2 Autorización

La autorización asegura que un usuario autenticado solo pueda realizar las acciones para las que está facultado. Se ejecuta verificando los niveles de privilegio incrustados y firmados criptográficamente en el token antes de responder a una petición.

2.4.3 Roles

El modelo de datos incluye una propiedad rígida de “Rol” (ej. Administrador, Cajero). Este mecanismo se implementa mediante *Middlewares* (bloques de código intermedios en el *backend*) que desenscriptan el *payload* del token y rechazan con un error 403 (Prohibido) cualquier intento de un Cajero de acceder a las rutas de eliminación de usuarios o reportes gerenciales.

2.4.4 Control de acceso

Tanto a nivel de la interfaz (ocultando botones de configuración a cajeros) como a nivel de capa de datos, se aplican políticas estrictas para evitar inyecciones maliciosas o robo de sesiones, blindando el flujo desde que se presiona “Cobrar” hasta que la información reposa en el disco.

2.5 Base de datos

El diseño de la persistencia de datos constituye el corazón del sistema, siendo responsabilidad directa de la ingeniería de datos modelar la información de la cafetería para que sea escalable, rápida y matemáticamente exacta.

2.5.1 Modelo relacional

Aunque tecnologías modernas como la pila MERN utilicen modelos NoSQL orientados a documentos para optimizar la velocidad transaccional, los principios lógicos relacionales son ineludibles en un TPS (Elmasri & Navathe, 2015). Las entidades maestras se identifican y separan claramente: Usuarios (Personal), Categorías (Clasificación del menú), Productos (Ítems de venta) y Facturas/Órdenes (Registro de la transacción). Se definen llaves referenciales explícitas entre ellas para establecer cardinalidad (ej. un cajero realiza muchas ventas, una orden contiene múltiples productos).

2.5.2 Integridad

Los principios lógicos de integridad se mantienen ineludibles mediante el uso de esquemas de validación estrictos (como *Mongoose* en el caso de la pila seleccionada). Estos esquemas garantizan la exactitud de los tipos de datos ingresados y previenen la inserción de documentos huérfanos o con información financiera incompleta.

2.5.3 Normalización

Se aplican reglas de normalización para evitar anomalías; por ejemplo, la información del perfil del usuario o la descripción detallada de un producto no se repiten innecesariamente. Sin embargo, por requerimientos de diseño de un POS y para proteger la contabilidad, se realiza una desnormalización controlada en las Órdenes: al registrar una venta, el precio exacto actual del producto se copia de forma fija dentro de la orden. Esto garantiza que la información histórica sea inmutable frente a futuros cambios de precios en el catálogo.

2.5.4 Transacciones

En el entorno TPS, una transacción es indivisible. Registrar una venta implica: calcular totales, insertar la orden, asociar el método de pago y actualizar la disponibilidad de la mesa. El motor de la base de datos se configura para garantizar atomicidad (Principios ACID), asegurando que si ocurre un fallo de red a la mitad del proceso, la base de datos ejecute un *Rollback* (reversión completa de los pasos previos), previniendo que existan “ventas a medias” o corrupciones en los arqueos de caja.

2.6 Metodología de desarrollo

2.6.1 Scrum

Es un marco de trabajo ágil para el desarrollo, entrega y mantenimiento de productos complejos, definido en la *Scrum Guide* (Schwaber & Sutherland, 2020). Se fundamenta en tres pilares empíricos: transparencia, inspección y adaptación. Para el Sistema POS de Cafetería, Scrum es la elección metodológica óptima porque permite iterar rápidamente y ajustar requerimientos de acuerdo a la retroalimentación del cliente.

Roles

- **Product Owner:** Es el responsable de maximizar el valor del producto. Sus funciones son definir y mantener el *Product Backlog*; ser el punto de contacto único

con el cliente; y aceptar o rechazar los incrementos funcionales.

- **Scrum Master:** Responsable de que el equipo aplique correctamente Scrum. Facilita las ceremonias, elimina impedimentos y protege al equipo de interrupciones externas.
- **Equipo de Desarrollo:** Equipo autoorganizado responsable de convertir los ítems del *Product Backlog* en un incremento potencialmente entregable (programación, diseño y pruebas).

Artefactos

- **Product Backlog:** Lista única y priorizada de todos los requerimientos funcionales y técnicos necesarios para el sistema.
- **Sprint Backlog:** Conjunto de requerimientos seleccionados para el *Sprint* actual, divididos en tareas concretas a desarrollar por el equipo.
- **Incremento:** La suma de todas las funcionalidades completadas durante el *Sprint*, las cuales deben cumplir con la Definición de Hecho (código probado y validado).

Eventos

- **Sprint Planning (Planificación):** Reunión de inicio donde el equipo define qué se va a entregar y cómo se va a construir el incremento durante el ciclo de trabajo.
- **Daily Scrum (Reunión diaria):** Reunión breve de sincronización del equipo de desarrollo para evaluar el progreso y exponer bloqueos u obstáculos.
- **Sprint Review (Revisión):** Demostración del *software* funcional al *Product Owner* y partes interesadas al finalizar el *Sprint* para recoger impresiones.
- **Sprint Retrospective (Retrospectiva):** Espacio de mejora continua donde el equipo reflexiona sobre sus propios procesos de trabajo de cara a la siguiente iteración.

Referencias Bibliográficas

- Elmasri, R., & Navathe, S. B. (2015). *Fundamentos de sistemas de bases de datos* (7th ed.). Pearson Educación.
- Laudon, K. C., & Laudon, J. P. (2020). *Sistemas de información gerencial: Administración de la empresa digital* (16th ed.). Pearson Educación.
- O'Brien, J. A., & Marakas, G. M. (2011). *Sistemas de información gerencial* (10th ed.). McGraw Hill.
- Schwaber, K., & Sutherland, J. (2020). *La guía definitiva de scrum: Las reglas del juego*. Scrum.org.
- Sommerville, I. (2015). *Ingeniería de software* (10th ed.). Pearson Educación.
- Stallings, W. (2017). *Fundamentos de seguridad en redes: Aplicaciones y estándares* (6th ed.). Pearson Educación.